

CineSpot

A web information system for movies showing in the cinemas of Kraków.

Project documentation

Course: Internet Technologies

Team G

Leonardo Miguel Fernandes Santos

Rustambek Rustamkhanov

Date: June 2026

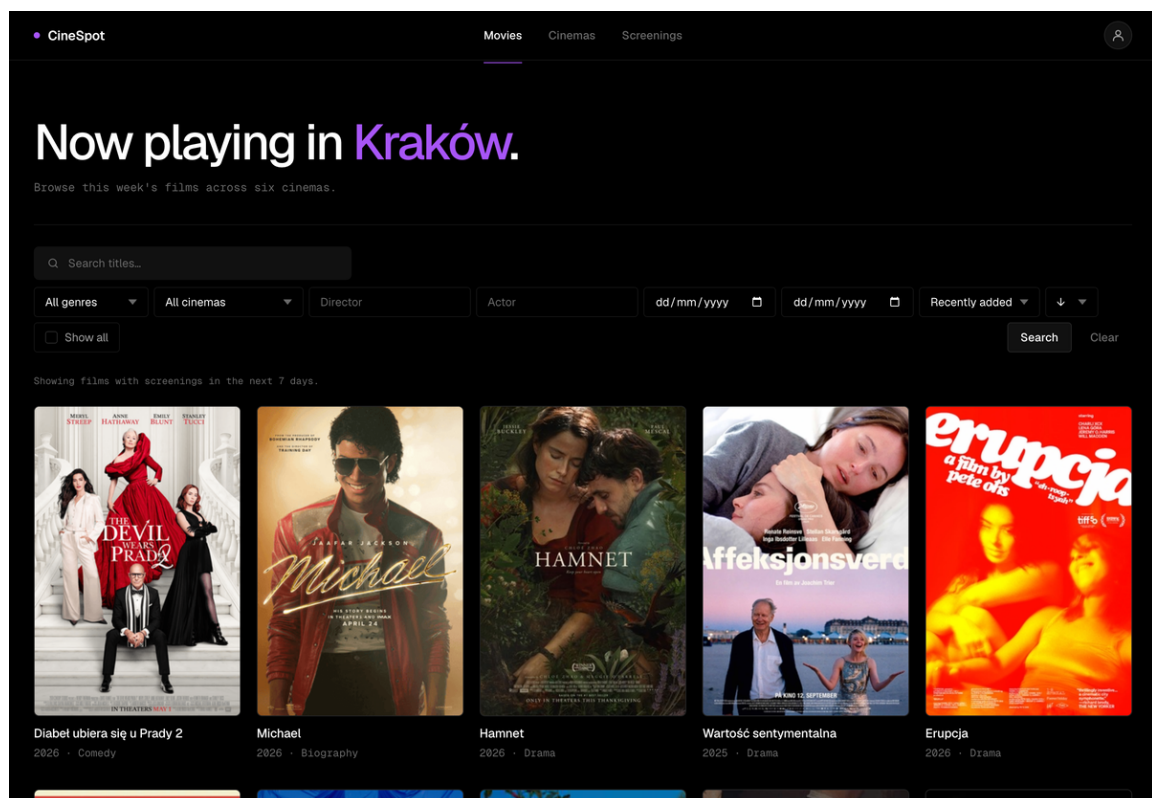


Figure 1. The home page: films currently showing in Kraków, with search, filters and sorting.

We wrote this to cover the project from a few sides. First what it does and the requirements it had to meet. Then the stack we chose, and how the app actually works inside: how the pages get built, how a user is logged in, where the data is kept. The last part is a tour of every screen, with the usual flows someone would follow.

1. Project topic

CineSpot shows what is playing in the cinemas of Kraków. A visitor can look through the films on right now, search and filter them, and open one to read its details and showtimes. Once you have an account you can rate films, keep a watchlist, and mark favourites, and not only favourite films but favourite actors and cinemas too. The administrator is the person who keeps the catalogue up to date (films, cinemas and screenings) and looks after the user accounts.

Our team (Team G) was given this subject: an information system about movies in Kraków cinemas, with three kinds of user, anonymous, logged in, and administrator.

2. Functional requirements

Anonymous user

- Browse the list of films that are currently showing.
- Search films by title and filter by genre, director, actor, cinema, release date and screening date.
- Sort the list by title, release date, genre, director or date added.
- Open a film to read its details and see its upcoming screenings (cinema, date, time, hall).
- Browse the list of cinemas and the full screenings schedule.
- See the average rating of each film.
- Register a new account and log in.

Authenticated user

- Everything an anonymous user can do, plus the following.
- Mark and unmark favourite films.
- Mark and unmark favourite actors, and see all films featuring those actors.
- Mark and unmark favourite cinemas, and see the upcoming screenings at those cinemas.
- Add films to a personal watchlist and remove them.
- Rate a film from 0 to 10, change the rating, or remove it.
- Log out.

Administrator

- Everything an authenticated user can do, plus the following.
- Create, edit and delete films (title, release date, genre, director, actors, description, poster).
- Create, edit and delete cinemas.
- Create, edit and delete screenings (a film, a cinema, a date and time, and a hall).
- Manage user accounts: change a role between user and admin, block or unblock an account, and delete an account.

3. Non-functional requirements

- **Security.** We never keep a password as plain text, only the bcrypt hash of it. Every query to the database is parameterised, so whatever a user types in a search box stays data and never becomes part of the SQL. Text that comes back from the database is escaped before it touches the page, which stops anyone slipping a script in through, say, a film title. And what you can do depends on who you are: some actions need you logged in, others are admin only.
- **Usability.** The whole thing is one page that never reloads, so moving around feels instant. Every action answers back with a small toast message, and the layout reflows to fit the window.
- **Performance.** SQLite runs in WAL mode and the queries are all prepared statements. For a catalogue this size that is more than fast enough.
- **Portability.** The database is just one file on disk, and there is no separate database server to set up. You clone the project, run npm install, and it starts.
- **Maintainability.** On the backend each resource has its own route file, and the database setup and the login guards sit on their own, so it is easy to find where a thing happens.

4. Technologies used

We built the backend on Node.js with Express, and kept the frontend deliberately plain: just HTML, CSS and JavaScript with a bit of jQuery. The two sides talk over AJAX, passing JSON back and forth. This is one of the stacks the professor allows for the final project, so we went with it instead of the Java and JSP route from the labs.

Layer	Technology	Role in the project
Runtime	Node.js 20	Runs the server.
Web framework	Express 5	Routing, request parsing, static files.
Sessions	express-session 1.19	Keeps the logged in user in a server side session, tied to a cookie.
Database	SQLite via better-sqlite3 12	Stores users, films, cinemas, screenings, favourites, watchlist and ratings.
Passwords	bcrypt 6	Hashes passwords on registration and checks them on login.
Frontend	HTML, CSS, JavaScript	A single page that renders the data it receives.
Frontend library	jQuery 3.7.1	DOM handling and AJAX calls to the API.

5. Architecture and project layout

Two halves, really. On one side the browser, which holds the single page and runs all the interface logic. On the other the server, sitting behind a JSON API under /api and also handing out the static files. We never build HTML on the server side. It just ships data, and it is the browser that turns that data into something you can actually look at. That one split is where most of the other decisions came from.

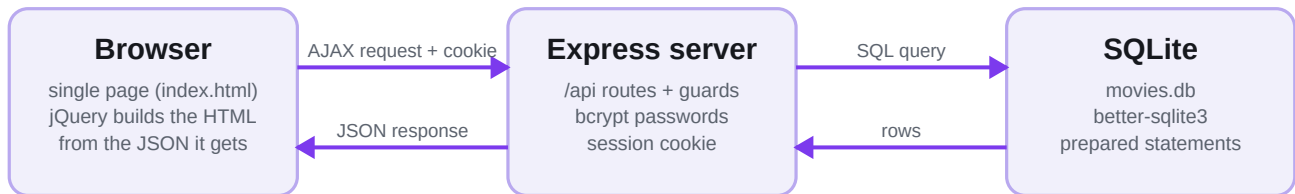


Diagram 1. How the browser, the server and the database talk to each other.

project/	
server.js	Express setup: parsing, session, static files, routes
config/database.js	opens SQLite, creates the tables and the default admin
middleware/auth.js	two guards: requireLogin and requireAdmin
routes/	
auth.js	register, login, logout, current session
movies.js	list, search, filters, details, screenings, CRUD
cinemas.js	list with search, CRUD (admin)
screenings.js	list with filters, CRUD (admin)
favourites.js	favourite films, actors and cinemas
watchlist.js	personal watchlist
ratings.js	average rating and the user's own rating
admin.js	user management
public/	
index.html	the single page and all its sections
css/style.css	styling
js/app.js	interface logic and the AJAX calls
seed.js	fills the database with films, cinemas, screenings

6. How the project works

6.1 How the views are generated

The whole interface is one file, index.html. Every section (films, cinemas, screenings, favourites, watchlist, admin) is just a block of HTML that we hide or show. Clicking the menu swaps which block is visible, and that is the only reason the page never reloads.

Those blocks start out nearly empty. When a section opens, the JavaScript fires an AJAX request, gets back a JSON array, and builds the HTML item by item, a film card here, a table row there. The grid of films on the home page, the favourites and the watchlist all run through the same render function.

```
$.get('/api/movies', params).done(function (movies) { // ask the API for data
  renderMovieGrid(movies, '#movie-grid');           // turn JSON into film cards
});
```

6.2 How users are authenticated

Registration first. The server validates the fields, checks that the username and the email are not already taken, runs the password through bcrypt, and stores the user. The password itself never gets written down, not in the database, not in a log, nowhere.

```
var hash = await bcrypt.hash(password, 10); // random salt, cost factor 10
db.prepare('INSERT INTO users (username, email, password_hash) VALUES (?, ?, ?)')
  .run(username.trim(), email.trim(), hash); // only the hash is stored
```

Logging in is the same thing backwards. The server looks the user up and hands the typed password to bcrypt to compare against the stored hash. The neat part is that bcrypt reads the salt straight out of that hash and recomputes from there, so it can check the password without ever undoing it. Match, and the user goes into the session.

```
var valid = await bcrypt.compare(password, user.password_hash);
if (!valid) return res.status(401).json({ error: 'Invalid username or password.' });
req.session.user = { id: user.id, username: user.username, role: user.role, ... };
```

The moment the session is set, express-session drops a signed cookie into the browser. The cookie holds an id and nothing more, no name, no role, none of that, and a signature made with a secret is what keeps anyone from forging one. After that the browser sends the cookie on every request, the server looks the matching session up, and it knows who is on the other end. We gave it eight hours before it expires. In front of the routes sit two small guards: one only asks that you are logged in (favourites, watchlist, ratings), the other insists you are an admin (the whole catalogue, plus the user management).

```
function requireLogin(req, res, next) {
  if (!req.session.user) return res.status(401).json({ error: 'You must be logged in.' });
  next();
}
function requireAdmin(req, res, next) {
  if (!req.session.user || req.session.user.role !== 'admin')
    return res.status(403).json({ error: 'Admin privileges required.' });
  next();
}
```

6.3 How the data is stored and collected

All the data sits in one SQLite file, movies.db. The first time the server runs it builds the tables and a default administrator (admin / admin123). The films do not come from any outside API. A seed script puts in 15 real films, 6 real Kraków cinemas, and a week of screenings. It only runs once, since it bails out if there are already films in there.

- **users:** account, role (user or admin) and status (active or blocked).
- **movies:** title, release date, genre, director, actors and poster.
- **cinemas:** name, address and city.
- **screenings:** links a film to a cinema with a date, time and hall.
- **favourites, favourite_actors, favourite_cinemas:** the three kinds of favourite, per user.
- **watchlist** and **ratings:** the personal watchlist and the 0 to 10 scores.

The screenings are scheduled for one fixed week, the 16th to the 22nd of June 2026, so during that week the home page always has films showing. Moving them to another week is a one line change: the base date at the top of the seed script. The schema diagram in 6.6 shows how all the tables fit together.

6.4 Security in practice

- Every query is a prepared statement with ? placeholders, so a value like an apostrophe or a SQL keyword in a search box stays plain text and cannot change the query.
- Before any text from the database is inserted into the page, the characters < > & and quotes are replaced by HTML entities, so a title cannot smuggle in a script.
- The user identity always comes from the server session, never from a value sent by the browser, so a user cannot act on behalf of someone else.

6.5 The API endpoints

Method	Path	What it does	Access
POST	/api/auth/register	Create an account.	public
POST	/api/auth/login	Log in and start a session.	public
POST	/api/auth/logout	End the session.	public
GET	/api/auth/session	Who is logged in.	public
GET	/api/movies	List films with search, filters and sorting.	public
GET	/api/movies/:id	One film and its screenings.	public
POST PUT DELETE	/api/movies	Create, edit, delete a film.	admin
GET	/api/cinemas	List cinemas with search.	public
POST PUT DELETE	/api/cinemas	Create, edit, delete a cinema.	admin
GET	/api/screenings	List screenings, filter by cinema and date.	public
POST PUT DELETE	/api/screenings	Create, edit, delete a screening.	admin
GET POST DELETE	/api/favourites...	Favourite films, actors and cinemas.	login
GET POST DELETE	/api/watchlist	Personal watchlist.	login
GET POST DELETE	/api/ratings/:movieId	Average rating and the user's score.	mixed
GET PUT DELETE	/api/admin/users	List users, change role or status, delete.	admin

So the round trip, start to finish: the browser fires an AJAX call and the cookie tags along. express-session turns that cookie back into the logged in user, the guard in front either lets it through or stops it, the route reads or writes SQLite, and the JSON answer makes its way back to the browser, which repaints only the bit of the screen that actually changed.

6.6 The database schema

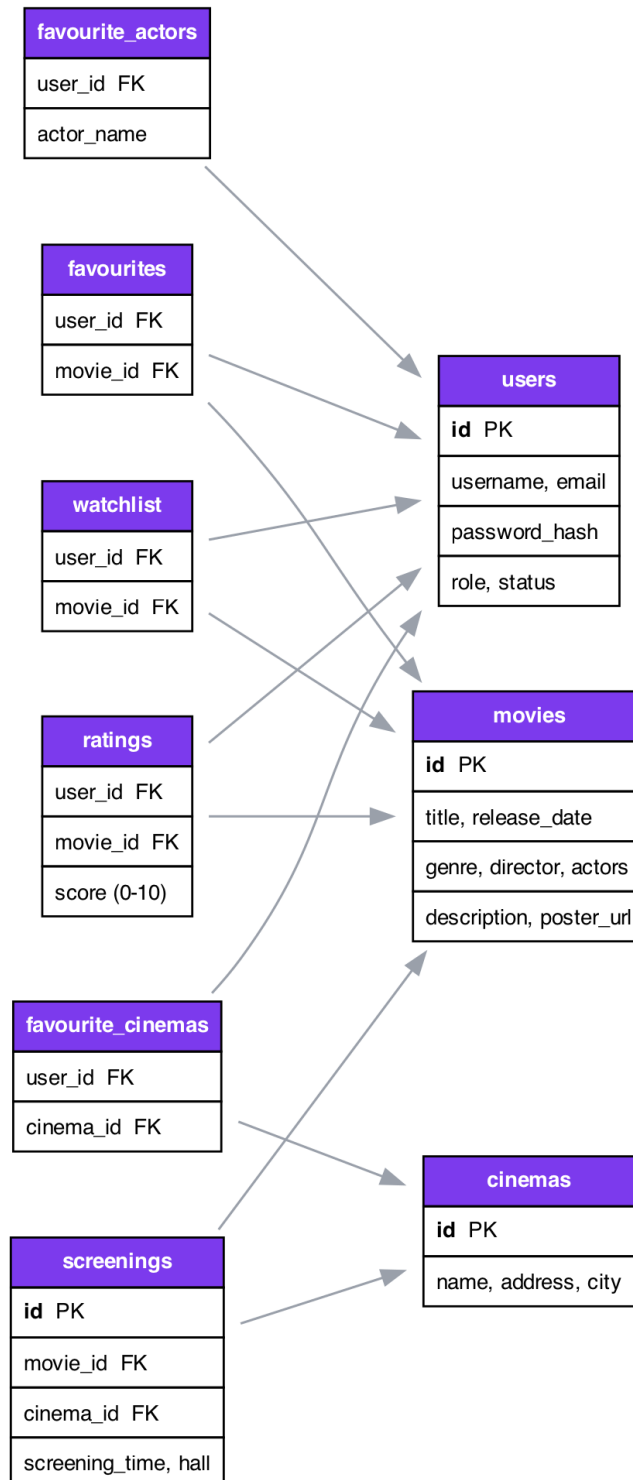


Diagram 2. Every table and the foreign keys that tie them together. An arrow points from a foreign key to the table it references. PK marks a primary key, FK a foreign key.

7. Visual walkthrough

Everything below is a screenshot from the app as it actually runs, nothing mocked up. The theme is dark, and since it is all one page, moving between sections happens with no wait at all.

7.1 Browsing and filtering films

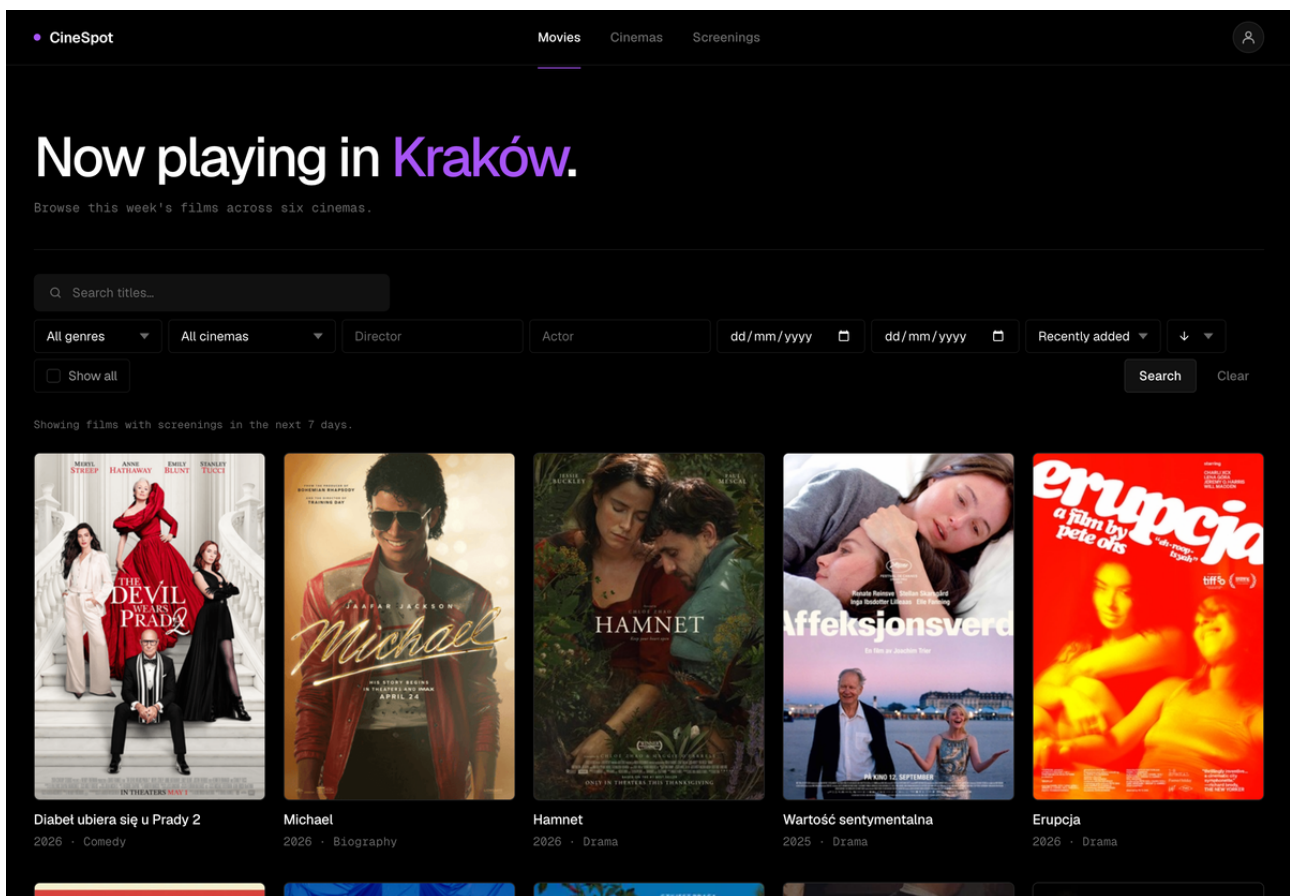


Figure 2. The home page. The top bar searches by title; the row below filters by genre, cinema, director, actor, screening date and release date, and sorts the results. By default the page shows only films that still have an upcoming screening; the 'Show all' toggle reveals every film in the catalogue.

7.2 Film details, screenings and rating



Movie poster for "The Devil Wears Prada 2" featuring Meryl Streep, Anne Hathaway, Emily Blunt, and Stanley Tucci.

Diabeł ubiera się u Prady 2

RATING: 9/10 (1 votes) YOUR RATING: 9

RELEASE DATE: 2026-05-01

GENRE: Comedy

DIRECTOR: David Frankel

ACTORS: Meryl Streep, Anne Hathaway, Emily Blunt, Stanley Tucci

DESCRIPTION: Długo wyczekiwana kontynuacja kultowej komedii o świecie mody.

UPCOMING SCREENINGS

2026-06-17 12:30	-	Cinema City Bonarka (Kraków)	-	Sala 1
2026-06-18 15:00	-	Kino Pod Baranami (Kraków)	-	Sala 1
2026-06-18 17:30	-	Kino Kijów (Kraków)	-	Sala Duża
2026-06-19 19:00	-	Multikino Kraków (Kraków)	-	Sala Kameralna
2026-06-20 17:30	-	Kino Pod Baranami (Kraków)	-	Sala 1
2026-06-20 19:00	-	Kino Kijów (Kraków)	-	Sala Duża
2026-06-20 20:30	-	Cinema City Bonarka (Kraków)	-	Sala 1
2026-06-21 20:30	-	Multikino Kraków (Kraków)	-	Sala Kameralna

Figure 3. A film opened by a logged in user. It shows the poster, the average rating and the user's own rating, the cast with a heart to favourite each actor, the description, and the list of upcoming screenings with cinema, date, time and hall.

7.3 Registration and login

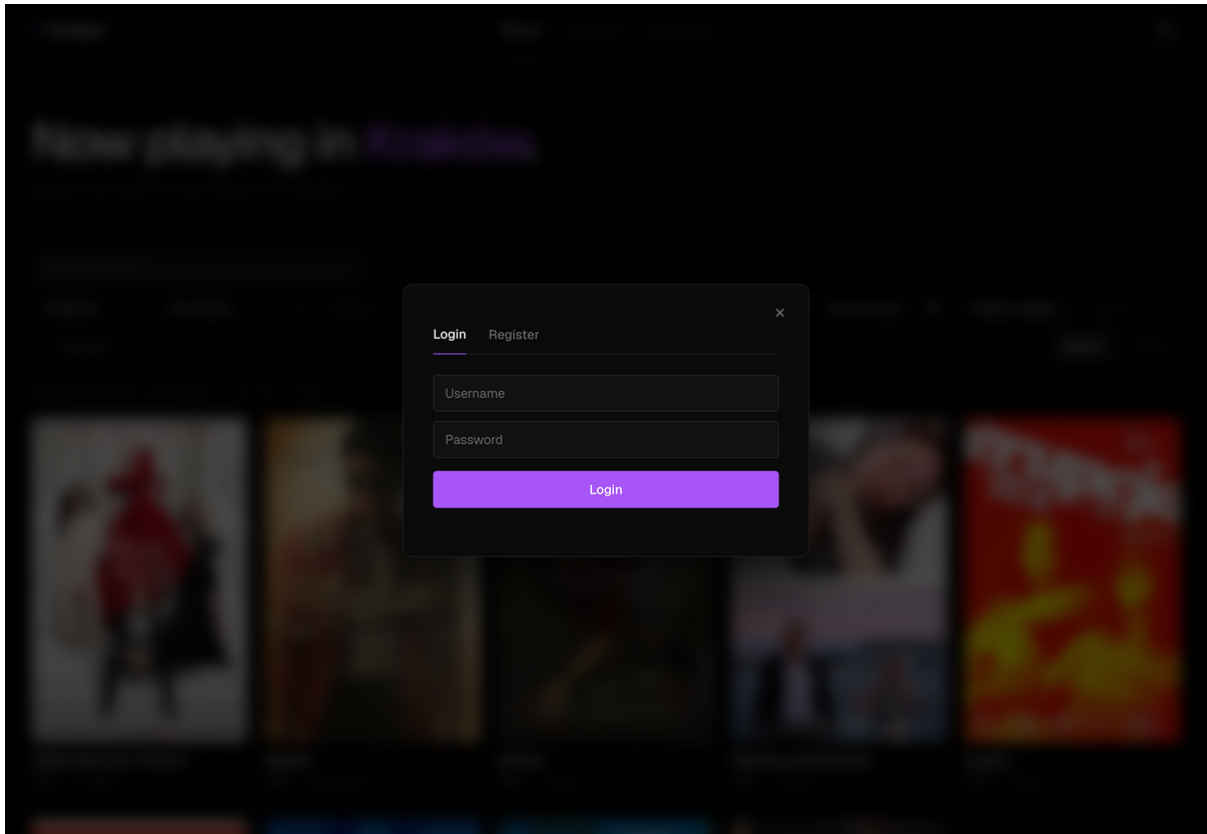


Figure 4. The login dialog. It opens over the page without a reload. Registration is the second tab.

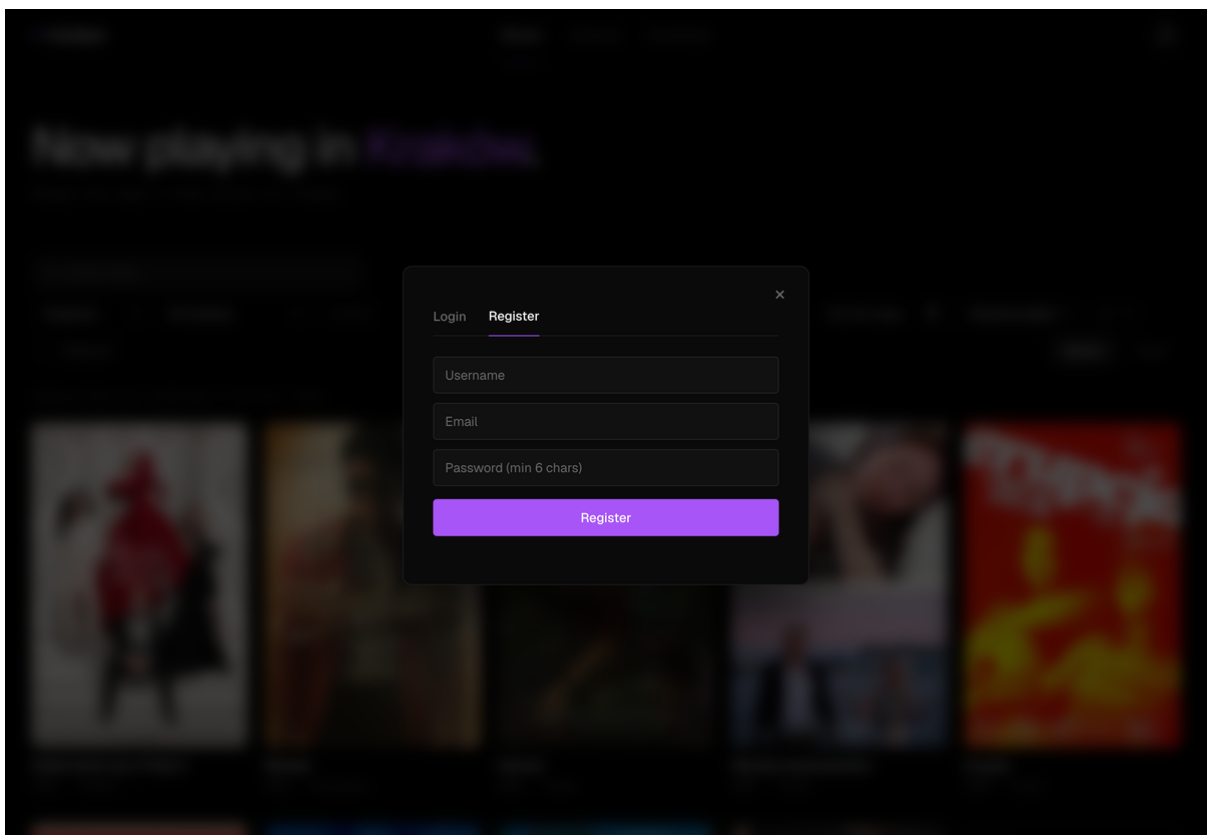


Figure 5. The registration form. The server checks the fields, refuses duplicate usernames or emails, and stores the password only as a bcrypt hash.

7.4 Favourites: films, actors and cinemas

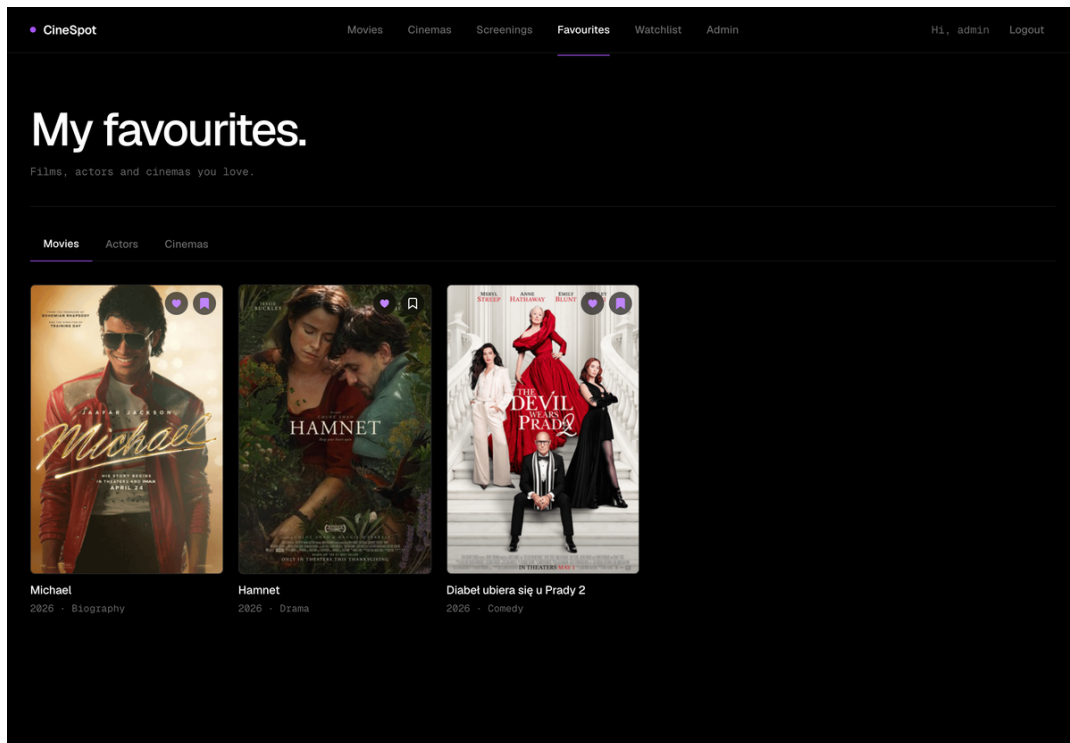


Figure 6. The favourites area has three tabs. The first lists the user's favourite films.

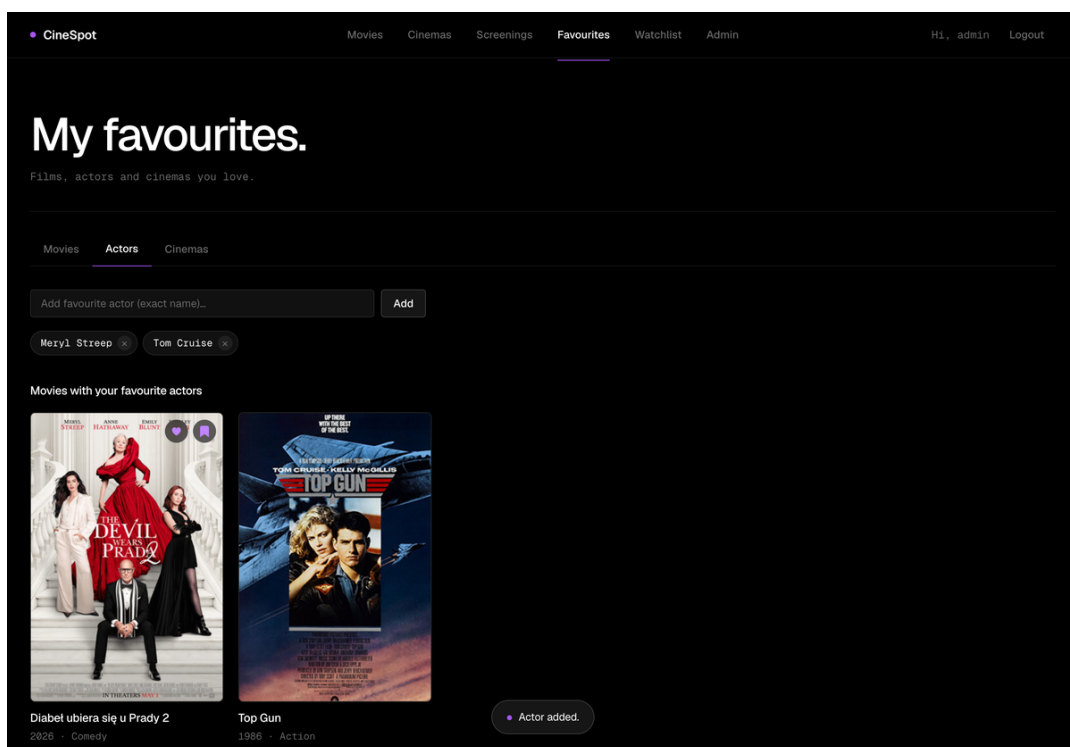


Figure 7. Favourite actors. The user adds an actor by name and the app lists every film in the catalogue featuring that actor.

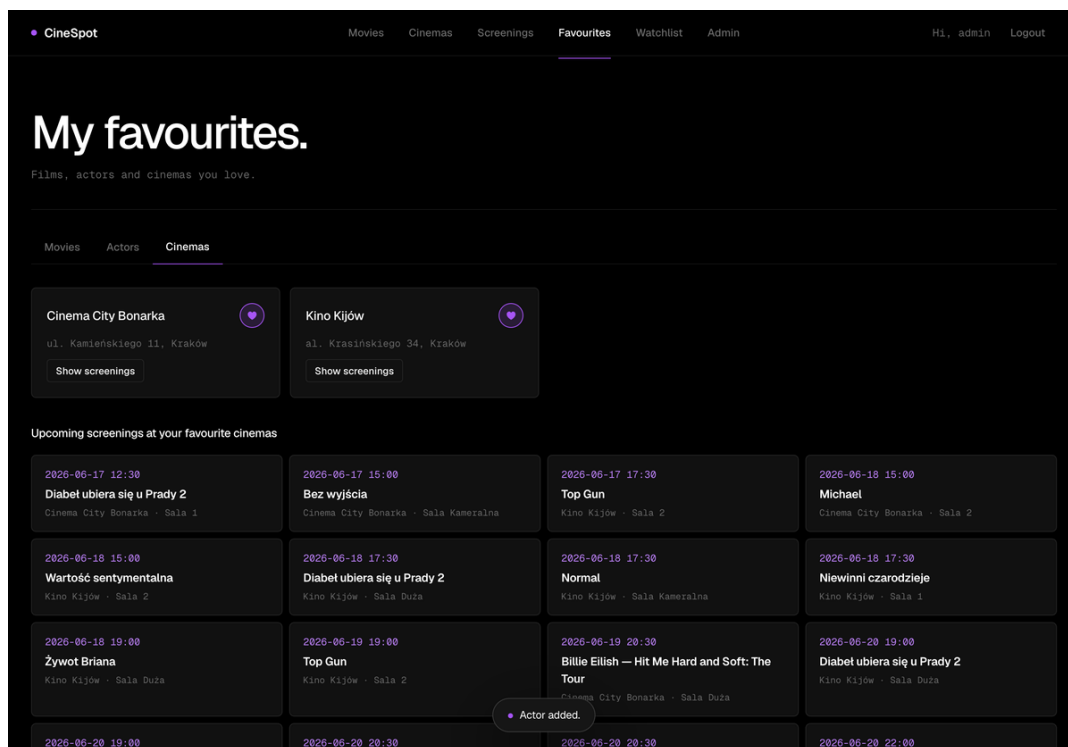


Figure 8. Favourite cinemas, together with the upcoming screenings at those cinemas.

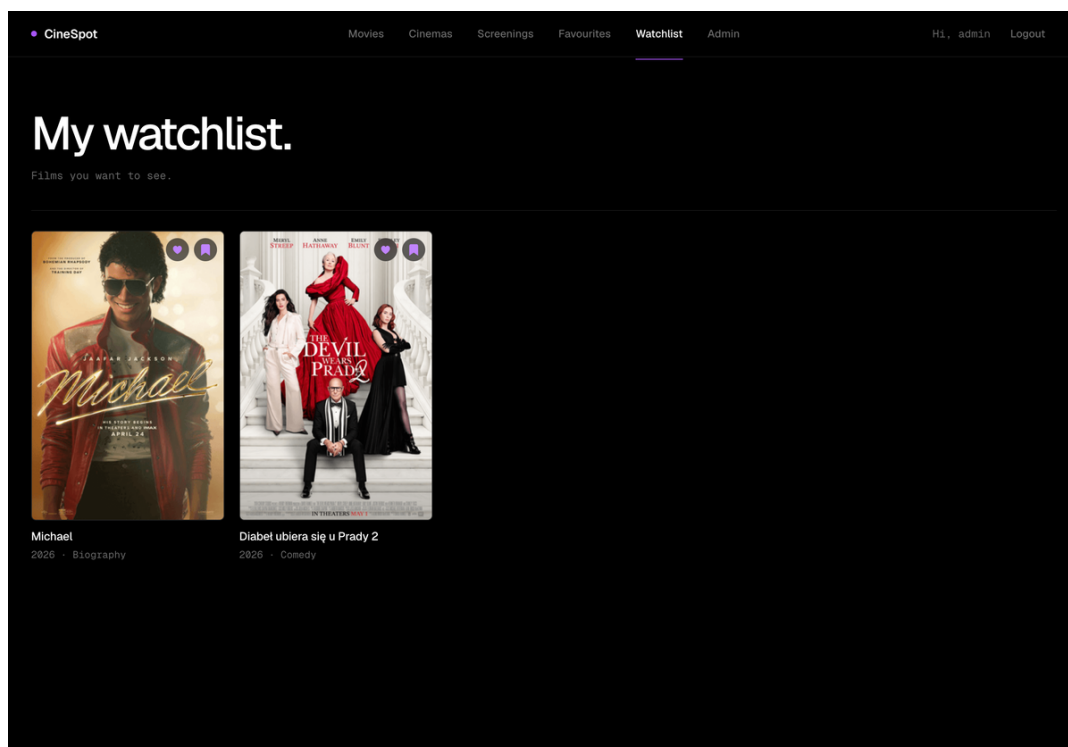


Figure 9. The personal watchlist: films the user has saved to watch later.

7.5 Cinemas and the schedule

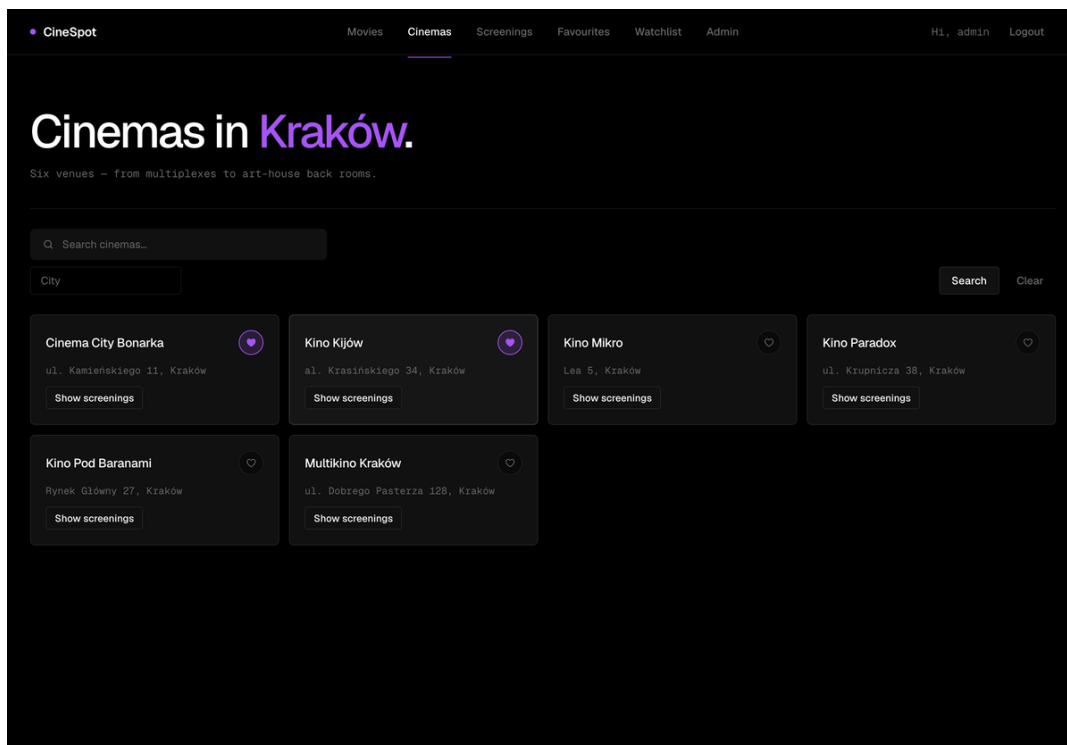


Figure 10. The cinemas page lists the six Kraków venues. A logged in user can favourite a cinema or jump to its screenings.

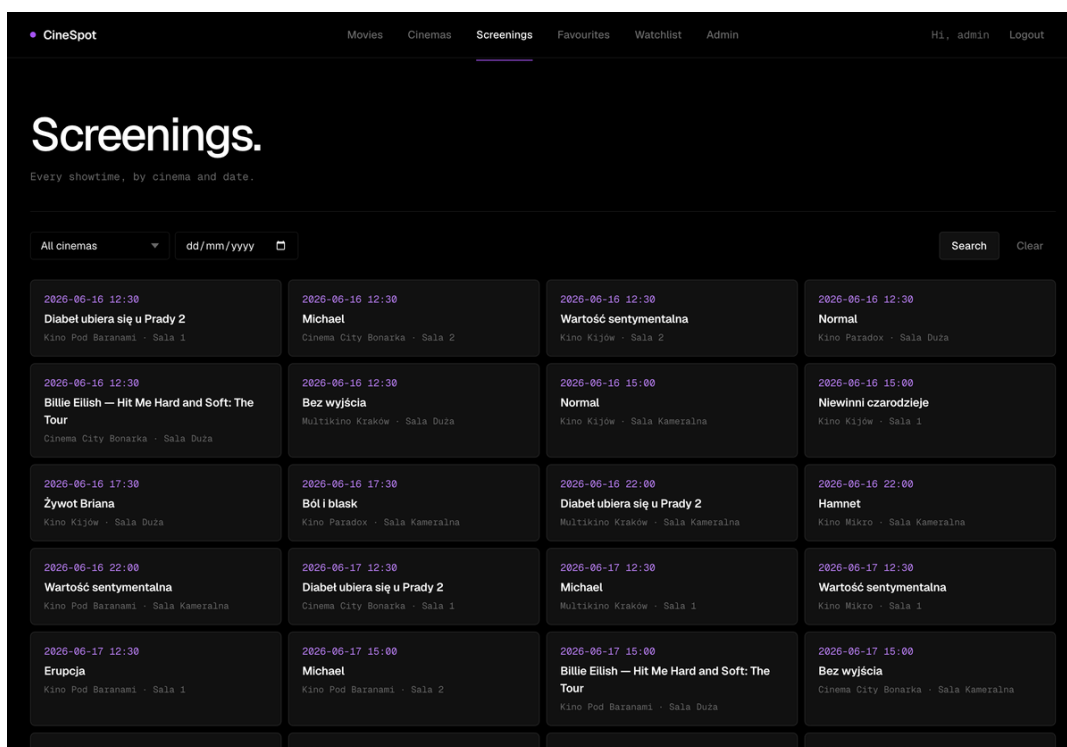


Figure 11. The screenings page shows every showtime, filtered by cinema and by date.

7.6 Administration

CineSpot

MoviesCinemasScreeningsFavouritesWatchlistAdmin

Hi, adminLogout

Admin.

Users, movies, cinemas, screenings.

Users

Movies

Cinemas

Screenings

ID	USERNAME	EMAIL	ROLE	STATUS	ACTIONS
3	rustambek	rustamkhanovrustambek1@gmail.com	USER	ACTIVE	Make adminBlockDelete
2	leonardo	leonardosantos1@gmail.com	USER	ACTIVE	Make adminBlockDelete
1	admin	admin@moviedb.local	ADMIN	ACTIVE	

Logged in as admin

Figure 12. User management. The administrator can change a role between user and admin, block or unblock an account, and delete it. The admin cannot delete its own account.

CineSpot

Movies

Cinemas

Screenings

Favourites

Watchlist

Admin

H1, adminLogout

Admin.

Users, movies, cinemas, screenings.

Users

Movies

Cinemas

Screenings

ID	TITLE	RELEASE DATE	GENRE	DIRECTOR	ACTIONS
1	Diabeł ubiera się u Prady 2	2026-05-01	Comedy	David Frankel	Edit Delete
2	Michael	2026-04-17	Biography	Antoine Fuqua	Edit Delete
3	Hamnet	2026-02-13	Drama	Chloé Zhao	Edit Delete
4	Wartość sentymentalna	2025-12-05	Drama	Joachim Trier	Edit Delete
5	Erupcja	2026-04-10	Drama	Peter Ohsa	Edit Delete
6	Normal	2026-04-24	Drama	Adele Tulli	Edit Delete
7	Billie Eilish — Hit Me Hard and Soft: The Tour	2026-05-08	Music	James Cameron, Billie Eilish	Edit Delete
8	Bez wyjścia	2026-03-20	Thriller	Daniel Jaroszek	Edit Delete
9	Wpatrując się w słońce	2025-11-21	Drama	Mascha Schilinski	Edit Delete
10	Pianista	2002-09-25	Drama	Roman Polański	Edit Delete

Figure 13. The film catalogue in the admin area, with edit and delete on each row.

Add Movie

Title

RELEASE DATE

dd/mm/yyyy

Genre

Director

Actors (comma separated)

Description

Poster URL

Save

Logged in as admin

Figure 14. The film form, used both to add a new film and to edit an existing one.

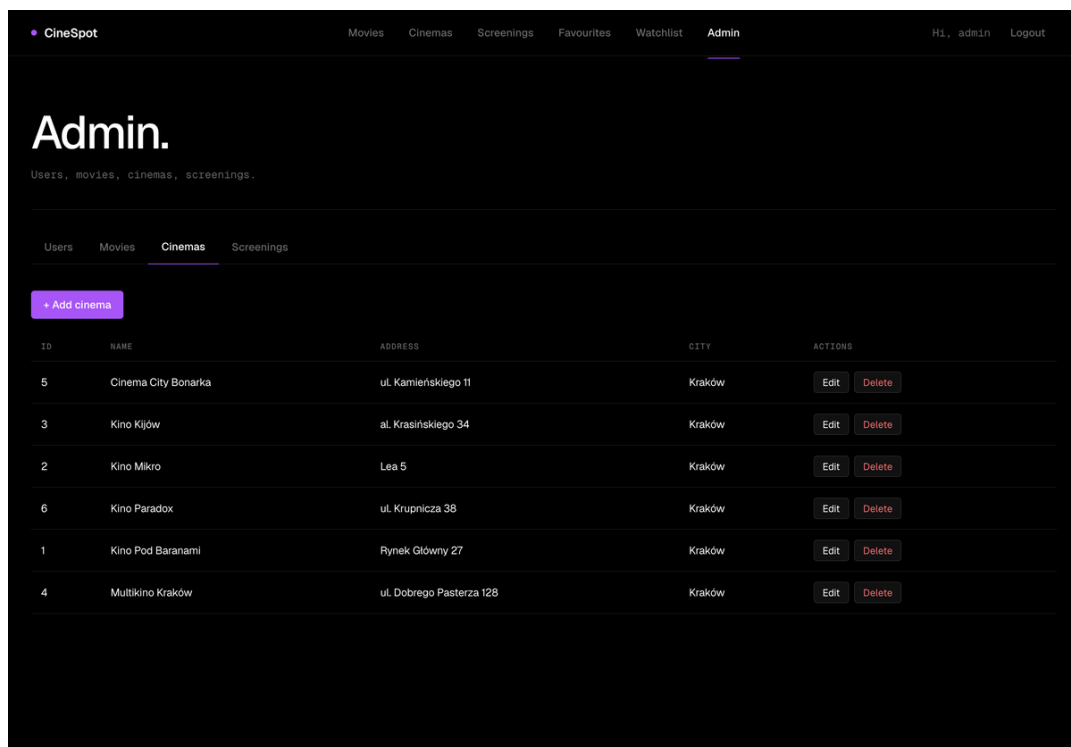


Figure 15. Cinema management.

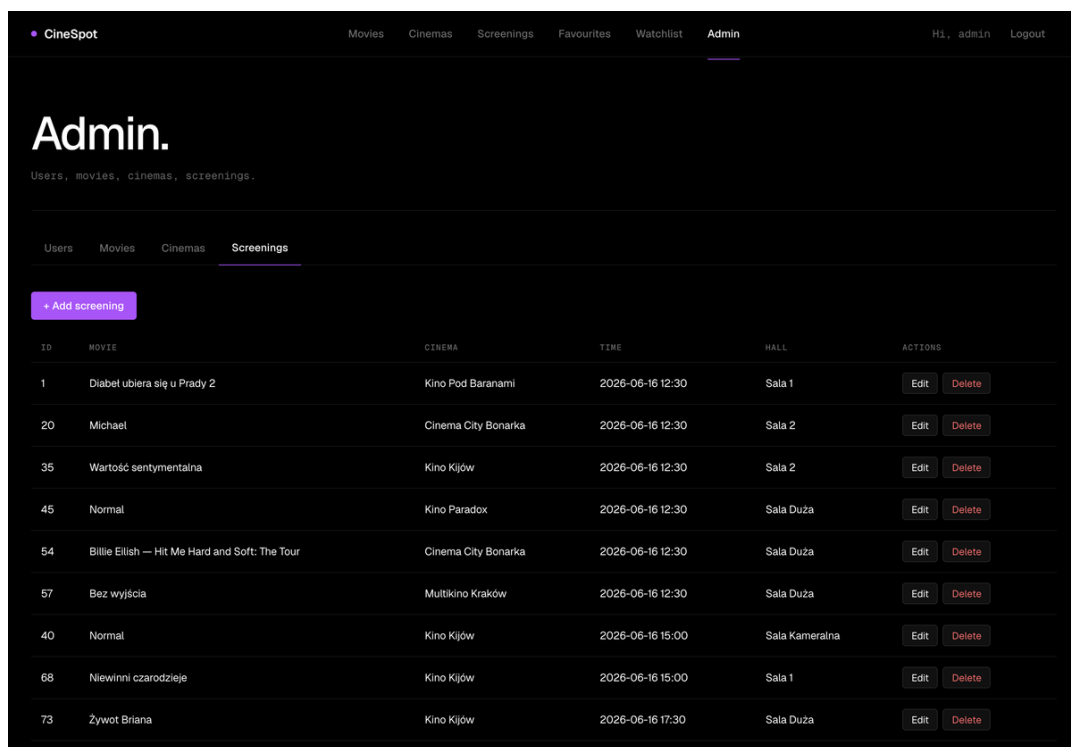


Figure 16. Screening management. Each screening links a film to a cinema with a date, time and hall.

8. Typical use case flows

Flow A. A visitor finds a film to watch tonight

- The visitor opens the site and sees the films currently showing.
- They pick a cinema and a date in the filters, or type part of a title.
- They open a film and read its details and its showtimes for the chosen cinema.

Flow B. A new user rates a film and saves favourites

- The user registers, then logs in. The greeting and the extra menus appear.
- They open a film and give it a score from 0 to 10. The average updates.
- They click the heart on the film, on an actor and on a cinema. The items appear in the favourites area.
- They add a film to the watchlist to watch it later.

Flow C. The administrator adds a film and schedules it

- The admin logs in and opens the admin area.
- In the films tab they add a new film with its details.
- In the screenings tab they create a screening, choosing the film, the cinema, the date and time, and the hall.
- The film now appears on the home page with its new screening.

9. How to run the project

```
npm install      # install Express, express-session, better-sqlite3, bcrypt
npm run seed     # first run only: fill the database with films, cinemas, screenings
npm start        # start the server at http://localhost:3003

# default administrator
user:  admin
pass:  admin123
```

On the first run the server builds the database and the default admin by itself.